VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

SEMINAR REPORT

CS163

Group members:

1. 25125052 - Võ Thành Đạt
2. 25125055 - Phạm Huy Khánh
3. 25125068 - Lê Phi Truyền
4. 25125089 - Lê Trọng Đại Nghĩa

24th March 2026

Contents

1 Introduction **2**

1.1 Overview of Quadtrees 2

1.2 Historical Background 2

2 Notation **3**

3 Programming Assignments **5**

3.1 Logical OR of Two Quad-Trees (LeetCode 558) 5

3.2 Construct Quad Tree (LeetCode 427) 9

3.3 Retrieve Cell Value in a Quad-Tree 13

3.4 Counting Necessary Nodes (Codeforces 2074F) 15

1 Introduction

1.1 Overview of Quadtrees

A **Quadtree** is a tree data structure in which each internal node has exactly four children. It is most often used to partition a two-dimensional space by recursively subdividing it into four quadrants or regions. The data represented may be points, regions, lines, or curves. Quadtrees are fundamental in fields that require spatial indexing, such as computer graphics, geographic information systems (GIS), and image processing. By using a hierarchical representation, Quadtrees allow for efficient spatial queries, reducing the time complexity of operations like range searching or collision detection from $O(N)$ to $O(\log N)$.

1.2 Historical Background

The term "Quadtree" was first coined by **Raphael Finkel** and **J.L. Bentley** in **1974** to describe a data structure for retrieving points based on their coordinates in a 2D plane. However, the concept of recursive decomposition of space has older roots in mathematics and cartography.
Shortly after its introduction, the structure was adapted into the **Region Quadtree** by **Hunter** in **1978** for image processing and computer graphics. During the 1980s and 1990s, new variants such as the *Point Quadtree*, *MX Quadtree*, and *PR Quadtree* were developed for different data types. Today, Quadtrees power many practical systems, including map tiling and collision detection.

2 Notation

- **Implementation:**
  - A Quad-Tree is implemented as a hierarchical tree structure where each internal node has exactly four children corresponding to four spatial quadrants.
  - Each node stores information about a specific region of the space. The node may either be a *leaf node*, representing a uniform region, or an *internal node* that subdivides the region into four smaller subregions.
  - The tree is typically constructed using a recursive divide-and-conquer strategy. If a region satisfies a stopping condition (for example, uniform data or minimum region size), a leaf node is created. Otherwise, the region is subdivided into four equal quadrants.
  - Each recursive step creates four child nodes representing the top-left, top-right, bottom-left, and bottom-right subregions.
  - Quad-Trees are commonly used for spatial partitioning problems such as image compression, collision detection, and geographic information systems.
- **Time Complexity:** $\mathcal{O}(n^2 \log n)$ where n is the side length of the grid.

  The analysis breaks down as follows:
  - In the worst case, the Quad-Tree needs to be fully constructed when no regions can be merged into leaves.
  - The recursion depth is $\log_2 n$ (since we divide the grid into quarters at each level).
  - At each level k of the recursion tree, there are 4^k subproblems.
  - Each subproblem at level k processes a grid of size $(n/2^k) \times (n/2^k) = \frac{n^2}{4^k}$ elements.
  - Total work at level k: $4^k \times \frac{n^2}{4^k} = n^2$.
  - Since there are $\log_2 n$ levels, the total time complexity is $\mathcal{O}(n^2 \log n)$.
- **Space Complexity:** $\mathcal{O}(n^2)$
  - **Recursion stack:** $\mathcal{O}(\log n)$ — The maximum depth of recursion is $\log_2 n$ when dividing the grid into quarters.

- **Quad-Tree nodes:** $\mathcal{O}(n^2)$ — In the worst case where no merging is possible, we create $\mathcal{O}(n^2)$ nodes (approximately $\frac{4^{\log n}-1}{3} = \frac{n^2-1}{3}$ internal nodes plus leaf nodes).
- The dominant factor is the space for storing the Quad-Tree nodes, resulting in $\mathcal{O}(n^2)$ total space complexity.

3 Programming Assignments

3.1 Logical OR of Two Quad-Trees (LeetCode 558)

Problem Description

This problem asks us to merge two $n \times n$ binary matrices represented by Quad-Trees using a logical OR operation.

A Quad-Tree is a tree data structure where each internal node has exactly four children representing the four quadrants: top-left, top-right, bottom-left, and bottom-right. Each node in the tree has two properties:

- `val`: A boolean value (True for 1s, False for 0s)
- `isLeaf`: A boolean indicating whether the node is a leaf (True) or has four children (False)

The construction follows these rules:

- **If a region contains all the same values** (all 0s or all 1s):
  - Create a leaf node with isLeaf = True
  - Set val to the grid's value (True for 1, False for 0)
  - The four children are set to null
- **If a region contains mixed values** (both 0s and 1s):
  - Create an internal node with isLeaf = False
  - Set val to any value (it doesn't matter since it's not a leaf)
  - Divide the current region into four equal quadrants
  - Recursively construct four children nodes for each quadrant

Solution Approach

This solution using recursion to construct the Quad-Tree. Here is how the solution works step by step.

Step 1: Base cases

```
if (!quadTree1 && !quadTree2) return nullptr;
if (!quadTree1) return quadTree2;
if (!quadTree2) return quadTree1;
if (quadTree1->isLeaf && quadTree1->val) return quadTree1;
if (quadTree2->isLeaf && quadTree2->val) return quadTree2;
if (quadTree1->isLeaf && quadTree2->isLeaf) return quadTree1;
```

- **Null checks:** If both tree is null, return null pointer. If either tree is null, the result simply inherits the remaining structure of the other tree.
- **Dominant All `1's` Leaves:** If either `quadTree1` or `quadTree2` is a leaf node with `val=1`, it dominates the OR operation.
- **False Leaves:** If both nodes are leaves but both value is false, then return either tree.

Step 2: Divide and merging

- If the base cases are not met, the current region must be subdivided.
- We continue call `intersect` and calculate on four quadrants (topLeft, topRight, bottomLeft, bottomRight)
- After the recursion returns from the four children, the current node evaluates them.
- If all four children are leaf nodes and they all share the exact same boolean value, the subdivision is redundant. Then we must convert the current internal node into a leaf node and removes the redundant child nodes.

Complexity Analysis

- **Time Complexity:** $\mathcal{O}(n^2 \log n)$ where n is the side length of the grid.
- **Space Complexity:** $\mathcal{O}(n^2)$

Implementation

```
/*
// Definition for a QuadTree node.
class Node {
public:
    bool val;
    bool isLeaf;
    Node* topLeft;
    Node* topRight;
    Node* bottomLeft;
    Node* bottomRight;

    Node() {
        val = false;
        isLeaf = false;
        topLeft = NULL;
        topRight = NULL;
        bottomLeft = NULL;
        bottomRight = NULL;
    }

    Node(bool _val, bool _isLeaf) {
        val = _val;
        isLeaf = _isLeaf;
        topLeft = NULL;
        topRight = NULL;
        bottomLeft = NULL;
        bottomRight = NULL;
    }

    Node(bool _val, bool _isLeaf, Node* _topLeft, Node* _topRight,
    Node* _bottomLeft, Node* _bottomRight) {
        val = _val;
        isLeaf = _isLeaf;
        topLeft = _topLeft;
        topRight = _topRight;
```

```
        bottomLeft = _bottomLeft;
        bottomRight = _bottomRight;
    }
};
*/

class Solution {
public:
    Node* intersect(Node* quadTree1, Node* quadTree2) {
    if (!quadTree1 && !quadTree2) return nullptr;
    if (!quadTree1) return quadTree2;
    if (!quadTree2) return quadTree1;
    if (quadTree1->isLeaf && quadTree1->val) return quadTree1;
    if (quadTree2->isLeaf && quadTree2->val) return quadTree2;
    if (quadTree1->isLeaf && quadTree2->isLeaf) return quadTree1;
    Node *cur = new Node(false, false);
    cur->topLeft = intersect(quadTree1->topLeft, quadTree2->topLeft);
    cur->topRight = intersect(quadTree1->topRight, quadTree2->topRight);
    cur->bottomLeft = intersect(quadTree1->bottomLeft, quadTree2->bottomLeft);
    cur->bottomRight = intersect(quadTree1->bottomRight,
        quadTree2->bottomRight);
    if (cur->topLeft && cur->topRight && cur->bottomLeft && cur->bottomRight) {
      if (cur->topLeft->isLeaf && cur->topRight->isLeaf
            && cur->bottomLeft->isLeaf && cur->bottomRight->isLeaf) {
        if (cur->topLeft->val == cur->topRight->val
                && cur->topLeft->val == cur->bottomLeft->val
        && cur->topLeft->val == cur->bottomRight->val) {
          cur->isLeaf = true;
          cur->val = cur->topLeft->val;
          cur->topLeft = cur->topRight =
                    cur->bottomLeft = cur->bottomRight = nullptr;
        }
      }
    }
    return cur;
    }
};
```

3.2 Construct Quad Tree (LeetCode 427)

Problem Description

This problem asks us to construct a Quad-Tree representation from an $n \times n$ binary matrix.
A Quad-Tree is a tree data structure where each internal node has exactly four children representing the four quadrants: top-left, top-right, bottom-left, and bottom-right. Each node in the tree has two properties:

- **val**: A boolean value (True for 1s, False for 0s)
- **isLeaf**: A boolean indicating whether the node is a leaf (True) or has four children (False)

The construction follows these rules:

- **If a region contains all the same values** (all 0s or all 1s):
  - Create a leaf node with `isLeaf = True`
  - Set `val` to the grid's value (True for 1, False for 0)
  - The four children are set to null
- **If a region contains mixed values** (both 0s and 1s):
  - Create an internal node with `isLeaf = False`
  - Set `val` to any value (it doesn't matter since it's not a leaf)
  - Divide the current region into four equal quadrants
  - Recursively construct four children nodes for each quadrant

Solution Approach

This solution uses a divide-and-conquer recursion to construct the Quad-Tree. Here is how the algorithm works step by step.

Step 1: Check if the current region is uniform

```
bool isSame(vector<vector<int>>& grid, int r, int c, int size){
    int val = grid[r][c];
    for(int i = r; i < r + size; i++){
        for(int j = c; j < c + size; j++){
            if(grid[i][j] != val) return false;
        }
    }
    return true;
}
```

- We check whether all cells in the current square region have the same value.
- If the region is uniform, we create a leaf node and stop recursion.

Step 2: Recursively divide the region

```
Node* build(vector<vector<int>>& grid, int r, int c, int size){
    if(isSame(grid, r, c, size)){
        return new Node(grid[r][c], true);
    }

    int half = size / 2;

    Node* topLeft = build(grid, r, c, half);
    Node* topRight = build(grid, r, c + half, half);
    Node* bottomLeft = build(grid, r + half, c, half);
    Node* bottomRight = build(grid, r + half, c + half, half);

    return new Node(true, false, topLeft, topRight, bottomLeft, bottomRight);
}
```

- If the region contains mixed values, we divide the square into four equal quadrants.
- We recursively construct Quad-Trees for each quadrant.
- The current node becomes an internal node with four children.

Complexity Analysis

- **Time Complexity**: $O(n^2 \log n)$ in the worst case because we may check each region multiple times during recursion.
- **Space Complexity**: $O(n^2)$ for the recursion tree and Quad-Tree nodes.

Implementation

```
/*
// Definition for a QuadTree node.
class Node {
public:
    bool val;
    bool isLeaf;
    Node* topLeft;
    Node* topRight;
    Node* bottomLeft;
    Node* bottomRight;

    Node() {
        val = false;
        isLeaf = false;
        topLeft = NULL;
        topRight = NULL;
        bottomLeft = NULL;
        bottomRight = NULL;
    }

    Node(bool _val, bool _isLeaf) {
        val = _val;
        isLeaf = _isLeaf;
        topLeft = NULL;
        topRight = NULL;
```

```
        bottomLeft = NULL;
        bottomRight = NULL;
    }

    Node(bool _val, bool _isLeaf, Node* _topLeft, Node* _topRight,
         Node* _bottomLeft, Node* _bottomRight) {
        val = _val;
        isLeaf = _isLeaf;
        topLeft = _topLeft;
        topRight = _topRight;
        bottomLeft = _bottomLeft;
        bottomRight = _bottomRight;
    }
};
*/

class Solution {
public:

    bool isSame(vector<vector<int>>& grid, int r, int c, int size){
        int val = grid[r][c];
        for(int i = r; i < r + size; i++){
            for(int j = c; j < c + size; j++){
                if(grid[i][j] != val) return false;
            }
        }
        return true;
    }

    Node* build(vector<vector<int>>& grid, int r, int c, int size){
        if(isSame(grid, r, c, size)){
            return new Node(grid[r][c], true);
        }
```

```

        int half = size / 2;

        Node* topLeft = build(grid, r, c, half);
        Node* topRight = build(grid, r, c + half, half);
        Node* bottomLeft = build(grid, r + half, c, half);
        Node* bottomRight = build(grid, r + half, c + half, half);

        return new Node(true, false, topLeft, topRight, bottomLeft, bottomRight);
    }

    Node* construct(vector<vector<int>>& grid) {
        int n = grid.size();
        return build(grid, 0, 0, n);
    }
};
```

3.3 Retrieve Cell Value in a Quad-Tree

Problem Description

Given a Quad-Tree that represents an $n \times n$ binary matrix and a specific coordinate (r, c), where $0 \leq r, c < n$, the task is to retrieve the value of the cell at that position without reconstructing the entire matrix.

The Quad-Tree uses the same `Node` structure:

- **isLeaf**: True if the node represents a uniform region.
- **val**: The value of the region (if it is a leaf).
- **children**: `topLeft`, `topRight`, `bottomLeft`, `bottomRight` (if it is not a leaf).

Solution Approach

Because a Quad-Tree is a spatial partition, we can use the coordinates (r, c) to decide which quadrant to move into at each step. This is a top-down traversal similar to searching in a Binary Search Tree, but with four branches.

Step 1: Base Case - Leaf Node

If the current node is a leaf (`isLeaf == true`), every cell within this quadrant has the same value. Therefore, the value at (r, c) is simply `node->val`.

Step 2: Recursive Step - Determine the Quadrant

If the node is not a leaf, we calculate the midpoint of the current region ($mid = size/2$) to determine which of the four children contains the target coordinate:

- If $r < mid$ and $c < mid$: Target is in **topLeft**.
- If $r < mid$ and $c \geq mid$: Target is in **topRight** (adjust c to $c - mid$).
- If $r \geq mid$ and $c < mid$: Target is in **bottomLeft** (adjust r to $r - mid$).
- If $r \geq mid$ and $c \geq mid$: Target is in **bottomRight** (adjust r to $r - mid$ and c to $c - mid$).

Complexity Analysis

- **Time Complexity**: $O(\log n)$ or $O(H)$, where H is the height of the tree. In the worst case, we travel from the root to a leaf at the maximum depth.
- **Space Complexity**: $O(\log n)$ due to the recursion stack.

Implementation

```
class Solution {
public:
    bool getCellValue(Node* root, int n, int r, int c) {
        // Base case: If it's a leaf, the value is uniform across the region
        if (root->isLeaf) {
            return root->val;
        }
```

```

        int mid = n / 2;

        // Determine which quadrant the coordinate (r, c) falls into
        if (r < mid && c < mid) {
            return getCellValue(root->topLeft, mid, r, c);
        }
        else if (r < mid && c >= mid) {
            // Adjust column index for the right-side quadrant
            return getCellValue(root->topRight, mid, r, c - mid);
        }
        else if (r >= mid && c < mid) {
            // Adjust row index for the bottom-side quadrant
            return getCellValue(root->bottomLeft, mid, r - mid, c);
        }
        else {
            // Adjust both indices for the bottom-right quadrant
            return getCellValue(root->bottomRight, mid, r - mid, c - mid);
        }
    }
};
```

3.4 Counting Necessary Nodes (Codeforces 2074F)

Problem Description

A **quadtree** is a tree data structure in which each node has at most four children and accounts for a square-shaped region.

Formally, **for all tuples** of nonnegative integers $k, a, b \geq 0$, there exists **exactly one node** accounting for the following region:

$$[a \cdot 2^k, (a+1) \cdot 2^k] \times [b \cdot 2^k, (b+1) \cdot 2^k]$$

All nodes whose region is larger than 1×1 contain four children corresponding to the regions divided equally into four, and the nodes whose region is 1×1 correspond to the leaf nodes of the tree.

The pink soldiers have given you a finite region $[l_1, r_1] \times [l_2, r_2]$, where l_i and r_i ($l_i < r_i$) are nonnegative integers. Please find the minimum number of nodes that you must choose in order to make the union of regions accounted for by the chosen nodes **exactly** the same as the given region. Here, two sets of points are considered different if there exists a point included in one but not in the other.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The only line of each test case contains four integers l_1, r_1, l_2, r_2 — the boundaries of the region in each axis ($0 \leq l_i < r_i \leq 10^6$).

Output

For each test case, output the minimum number of nodes necessary to satisfy the condition on a separate line.

Example

Input:

```
5
0 1 1 2
0 2 0 2
1 3 1 3
0 2 1 5
9 98 244 353
```

Output:

```
1
1
```

```
4
5
374
```

Solution Approach

Notice how the intervals on each axis correspond to those found in nodes of standard 1D segment trees. Let us find the segment tree intervals required to cover the query range on each axis individually, and call those sets of intervals I_x and I_y.

Then, for each rectangular region formed by some $a \times b$ such that $a \in I_x$ and $b \in I_y$, the following properties hold:

1. No quadtree node with side length strictly greater than $\min(|a|, |b|)$ can cover this rectangle without spilling outside the given total region.
2. All quadtree nodes with side length no greater than $\min(|a|, |b|)$ are either completely inside the rectangle or completely outside.

Thus, we find the following strategy to cover the region optimally: For all rectangles $a \times b$ such that $a \in I_x$ and $b \in I_y$, cover the sub-region using the largest possible valid nodes. Since both $|a|$ and $|b|$ are powers of two (as they are segment tree nodes), the larger side length is perfectly divisible by the smaller side length.

Therefore, we can completely tile the $a \times b$ sub-region with square nodes of side length $\min(|a|, |b|)$. The number of square nodes required to cover this specific sub-region is:

$$\frac{\max(|a|, |b|)}{\min(|a|, |b|)}$$

Summing this value over all pairs of $a \in I_x$ and $b \in I_y$ gives the final answer. Implementing this leads to a solution with time complexity $\mathcal{O}(\log^2 X)$, where X is the maximum coordinate value. While faster $\mathcal{O}(\log X)$ solutions exist, they are not strictly necessary to pass the time limits for this problem.

Implementation

The following is an implementation of the logic described above. It recursively finds the optimal 1D segments for both axes and then computes the required quadtree squares by pairing them up.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int t;
    cin >> t;
    for (int i = 0; i < t; i++) {
        int l1, r1, l2, r2;
        cin >> l1 >> r1 >> l2 >> r2;
        vector<pair<ll, ll>> it1, it2;

        auto rec = [&](auto rec, int L, int R, int l, int r,
                       vector<pair<ll, ll>>& v) -> void {
            if (r <= L || l >= R) return;
            if (l <= L && R <= r) {
                v.emplace_back(L, R);
                return;
            }
            rec(rec, L, (L + R) / 2, l, r, v);
            rec(rec, (L + R) / 2, R, l, r, v);
        };

        rec(rec, 0, 1 << 25, l1, r1, it1);
        rec(rec, 0, 1 << 25, l2, r2, it2);

        ll ans = 0;

            for (auto [al, ar] : it1) {
            for (auto [bl, br] : it2) {
```

```
                ll a = ar - al;
                ll b = br - bl;
                if (a < b) swap(a, b);
                ans += a / b;
            }
        }
        cout << ans << "\n";
    }
    return 0;
}
```